

Part 1 — Foundations: the package, the data and the audit

Complete tutorial for sciModelFlowR (version 1.0.2)

Walter Esfrain Pereira

2026-09-19

Table of contents

1	How to use this tutorial	3
1.1	Installation	4
2	The package on one page	5
3	Simulated data with planted discrepancies	9
3.1	The field trial: outliers, missing values and heteroscedasticity	10
3.2	Two outcomes: an imbalanced class and a superdispersed count	12
3.3	Structure: groups, time, space and high dimension	14
4	Audit and contracts	19
4.1	Finding the planted outliers	21
4.2	Contracts: what the package refuses	22
5	Part 1 exercises	24

1 How to use this tutorial

This material walks through `sciModelFlowR` from beginning to end, in the order a real scientific project happens: **ask, declare, audit, protect, model, compare, explain, measure uncertainty and report**. It was written for someone who has never used the package and wants to learn both the syntax and the reasoning.

The central difference from an ordinary tutorial: **the data are simulated with known truth and deliberately planted discrepancies** (outliers, missing values, heteroscedasticity, imbalance, clustering, autocorrelation). Every method can therefore be judged against what was planted — not against a vague expectation.

i Conventions

- Every code block was **executed** on version 1.0.2; the numbers in the prose come from the code itself (never typed by hand).
- Figures and axes use English labels (scientific convention); explanations and captions are in English.
- Numbers in prose and tables use dot decimals, matching the figures.
- Coloured boxes mean: **Interpretation** (how to read the result), **Trap** (common mistake), **Good practice**, **In your experiment** (transfer to real data) and **Exercise**.

The tutorial has six parts:

Part	Theme	Package modules
1	Foundations: the package, the data and the audit	<code>doctor</code> , <code>datasets</code> , <code>data-audit</code> , <code>design</code>
2	Design, resampling and preparation	<code>resampling</code> , <code>bootstrap</code> , <code>preprocess</code> , <code>features</code> , <code>imbalance</code>
3	Supervised modeling and probabilities	<code>model-adapters</code> , <code>experiment</code> , <code>diagnostics</code> , <code>probabilistic</code>
4	Tuning, benchmarking, calibration and thresholds	<code>tuning</code> , <code>benchmark</code> , <code>calibration</code>
5	Explainability and uncertainty	<code>explain</code> , <code>conformal</code> , <code>uncertainty</code>
6	Advanced and production: Bayes, DL, persistence and reporting	<code>bayesian</code> , <code>deep-learning</code> , <code>tracking</code> , <code>persistence</code> , <code>reporting</code>

1.1 Installation

```
# Install the published version, with vignettes:  
remotes::install_github("wep69/sciModelFlowR", build_vignettes = TRUE)  
library(sciModelFlowR)
```

2 The package on one page

`sciModelFlowR` organises a modeling project as a **declared contract**: task, data, design, preprocessing, resampling, model and metrics are explicit objects. From there, the package **refuses** shortcuts that would produce optimistic numbers (for example, preprocessing before splitting, or tuning on the test set). Environment diagnostics is the first command of any session.

```
doctor <- smf_doctor()
cat("components checked:", nrow(doctor),
    "| available:", sum(doctor$available), "\n")
```

```
components checked: 31 | available: 31
```

```
tab_smf(subset(doctor, component %in% c("R", "sciModelFlowR", "stats", "torch", "xgboost", "brms"),
    caption = "Environment diagnostics: core components and optional backends.")
```

Table 2.1: Environment diagnostics: core components and optional backends.

	component	available	version	status
1	R	TRUE	4.6.0	core
2	sciModelFlowR	TRUE	1.0.2	core
13	xgboost	TRUE	3.2.1.1	optional-available
16	torch	TRUE	0.17.0	optional-available
19	brms	TRUE	2.23.0	optional-available

The package ships 25 synthetic validation datasets (“gold”), useful for studying expected behaviour before using your own data. Each one has a **card** describing the scenario, variables, known truth and pedagogical traps.

```
card <- smf_dataset_card("gold_linear_regression")
cat(substr(card, 1, 480), "...\\n")
```

```
# gold_linear_regression
```

```
## Scientific scenario
```

Synthetic agronomic yield response with independent observational units and a Gaussian linear data generating process. The dataset is intended to validate schema handling, holdout splitting, training, only preprocessing, reference linear-model adapters, prediction metrics, serialization, and reporting.

```
## Variables
```

```
- `obs_id`: stable observation identifier.  
- `nitrogen`: nitrogen dose, arbitrary teaching units ...
```

The public API has 246 functions, organised in 10 categories. The map below is the best way to know “where each topic lives”.

```
api <- smf_api_catalog()  
by_category <- as.data.frame(table(api$category), stringsAsFactors = FALSE)  
names(by_category) <- c("category", "functions")  
by_category <- by_category[order(-by_category$functions), ]  
tab_smf(by_category, caption = "Exported functions per API category (1.0.2).")
```

Table 2.2: Exported functions per API category (1.0.2).

	category	functions
1	core	88
6	modeling	35
5	inference-uncertainty	24
3	deep-learning	23
9	resampling-features	23
7	operations	17
8	reporting-validation	11
10	tuning-benchmark	10
2	data-design	9
4	explainability	6

```
ggplot(by_category, aes(stats::reorder(category, functions), functions)) +  
  geom_col(fill = cores_smf[["primaria"]], width = .7) +  
  geom_text(aes(label = functions), hjust = -0.2, size = 3.2) +  
  coord_flip() +  
  labs(x = NULL, y = "exported functions",  
        title = "API map: 246 exports by category") +  
  ylim(0, max(by_category$functions) * 1.12)
```

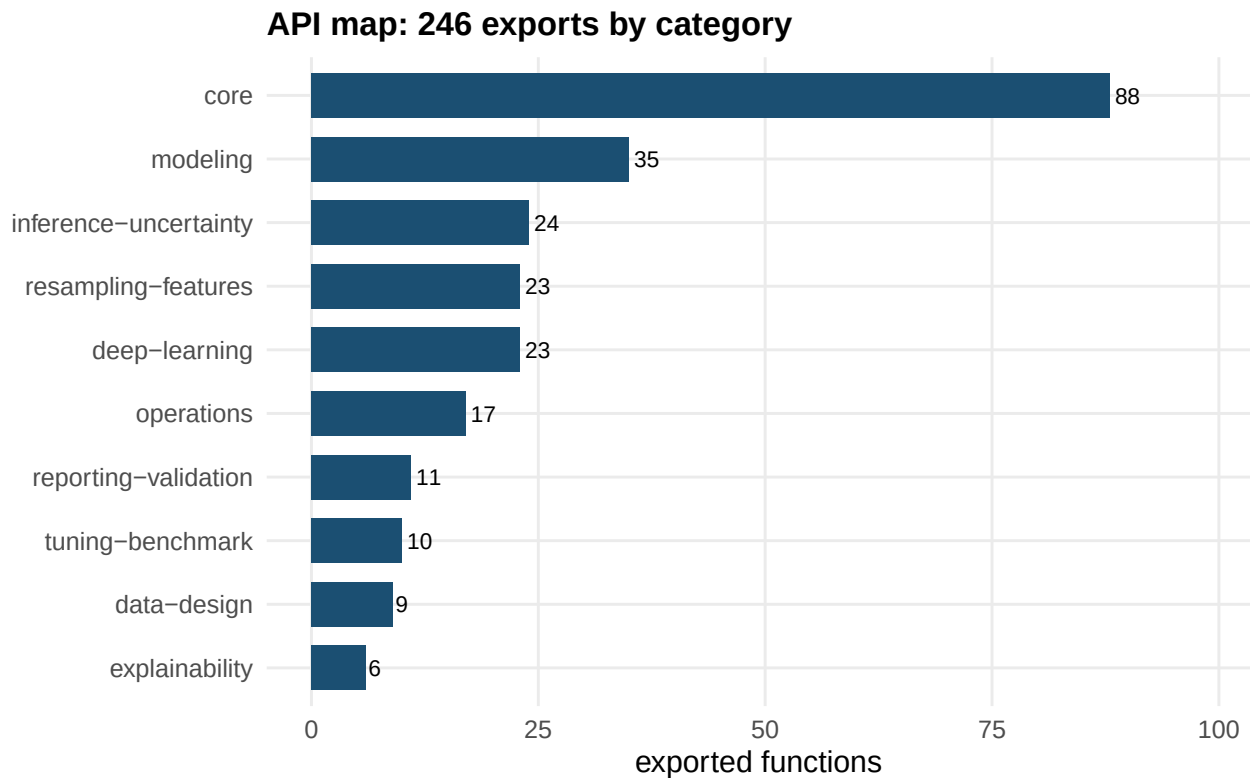


Figure 2.1: Distribution of the 246 exported functions across API categories.

Optional backends are declared, not presumed: the matrix below says what is core and what depends on an external package, and the diagnostics show what is available **on this machine**.

```
matrix_tbl <- smf_backend_matrix()
tab_smf(matrix_tbl[, c("backend", "tier", "package", "scope")],
        caption = "Backend matrix: core and optional.")
```

Table 2.3: Backend matrix: core and optional.

backend	tier	package	scope
stats	core-reference	base R	lm/glm differential reference
tidymodels	optional	parsnip/workflows	supervised modeling
mlr3	optional	mlr3	supervised modeling
xgboost	optional	xgboost	tree boosting
torch	optional	torch	Deep Learning
luz	optional	luz	DL trainer
keras3	optional	keras3	DL alternate backend
brms	optional	brms/cmdstanr	Bayesian multilevel/distributional
dbarts	optional	dbarts	BART
DiceKriging	optional	DiceKriging	Gaussian process
probably	optional	probably	calibration/conformal
mlflow	optional	mlflow	tracking

backend	tier	package	scope
pins	optional	pins	artifact registry
vetiver	optional	vetiver	deployment

```
state <- smf_capabilities()
backend_summary <- data.frame(
  backend = names(state),
  status = vapply(state, function(x) x$status, character(1))
)
tab_smf(backend_summary, caption = "State of each backend in the current environment.")
```

Table 2.4: State of each backend in the current environment.

	backend	status
brms	brms	advanced
dbarts	dbarts	advanced
DiceKriging	DiceKriging	advanced
keras3	keras3	advanced
luz	luz	advanced
mlr3	mlr3	advanced
probably	probably	advanced
stats	stats	core
tidymodels	tidymodels	advanced
torch	torch	advanced
xgboost	xgboost	advanced

💡 Good practice

Run `smf_doctor()` at the start of every session and keep the output in the report. “It worked on my machine” stops being an excuse when the diagnostics are recorded together with the result.

3 Simulated data with planted discrepancies

Nine datasets were simulated to cover the whole modeling cycle. The table is the **answer key**: it says what was planted and what the truth is. Each part of the tutorial uses a subset and compares what the method recovered with what is here.

```
gab <- gabarito
gab$detalhe <- substr(gab$detalhe, 1, 78)
names(gab) <- c("Dataset", "n", "p", "Discrepancy", "What was planted", "Truth")
tab_smf(gab[, c("Dataset", "n", "p", "Discrepancy", "What was planted")],
        caption = "Answer key for the nine simulated datasets (full detail in dados/gabarito.csv).")
```

Table 3.1: Answer key for the nine simulated datasets (full detail in dados/gabarito.csv).

Dataset	n	p	Discrepancy	What was planted
field_yield	180	8	outliers	5 parcelas (P023, P067, P101, P145, P177) deslocadas entre -3000 e +4100
field_yield	180	8	ausencias	11 NA em nitrogenio (MCAR 6%); 16 em organic_matter (MAR: bloco B4)
field_yield	180	8	heterocedasticidade	sigma do erro = 100 + 1.5 * nitrogenio
field_yield	180	8	efeito de bloco	efeitos B2=+120, B3=-80, B4=+60 kg/ha
dis-ease_binary	400	6	desbalanceamento	13.8% de casos positivos
dis-ease_binary	400	6	ruido de rotulo	20 plantas com rotulo invertido (5%)
farm_groups	240	4	agrupamento	24 fazendas, 10 talhoes cada; intercepto aleatorio sd=300
climate_series	730	4	choques	5 dias com deslocamento de +5,5 a +7,2 graus
soil_spatial	200	4	autocorrelacao espacial	covariancia exponencial: patamar 0,6; alcance 0,25; pepita 0,15
spectral_pn	120	202	alta dimensao	200 comprimentos de onda para 120 amostras; multicolinearidade severa
spectral_pn	120	202	outliers espectrais	4 espectros (A011, A047, A083, A109) deslocados em +0,35
pest_counts	150	4	superdispersao	binomial negativa com theta = 2 (variancia > media)
pest_counts	150	4	zeros extras	22 zeros plantados alem do modelo (15%)

Dataset	n	p	Discrepancy	What was planted
dl_tiny	300	4	nao linearidade	superficie com seno, quadrado e interacao; sem outlier
bayes_small	30	2	amostra pequena	n = 30; prior informativo conhecido para o conjugado

3.1 The field trial: outliers, missing values and heteroscedasticity

```
field <- ler("field_yield")
field$origin <- ifelse(field$plot_id %in% verdade$field_yield$outlier_ids,
                      "planted outlier", "regular plot")
cat("plots:", nrow(field),
    "| NA in nitrogen:", sum(is.na(field$nitrogen)),
    "| NA in organic_matter:", sum(is.na(field$organic_matter)), "\n")
```

plots: 180 | NA in nitrogen: 11 | NA in organic_matter: 16

```
ggplot(field, aes(nitrogen, yield)) +
  geom_point(aes(colour = origin), size = 2.2, alpha = .85) +
  scale_colour_manual(values = c("regular plot" = cores_smf[["primaria"]],
                                "planted outlier" = cores_smf[["alerta"]])) +
  labs(x = "Nitrogen (kg/ha)", y = "Yield (kg/ha)", colour = NULL,
       title = "Field trial: planted outliers") +
  theme(legend.position = "top")
```

Field trial: planted outliers

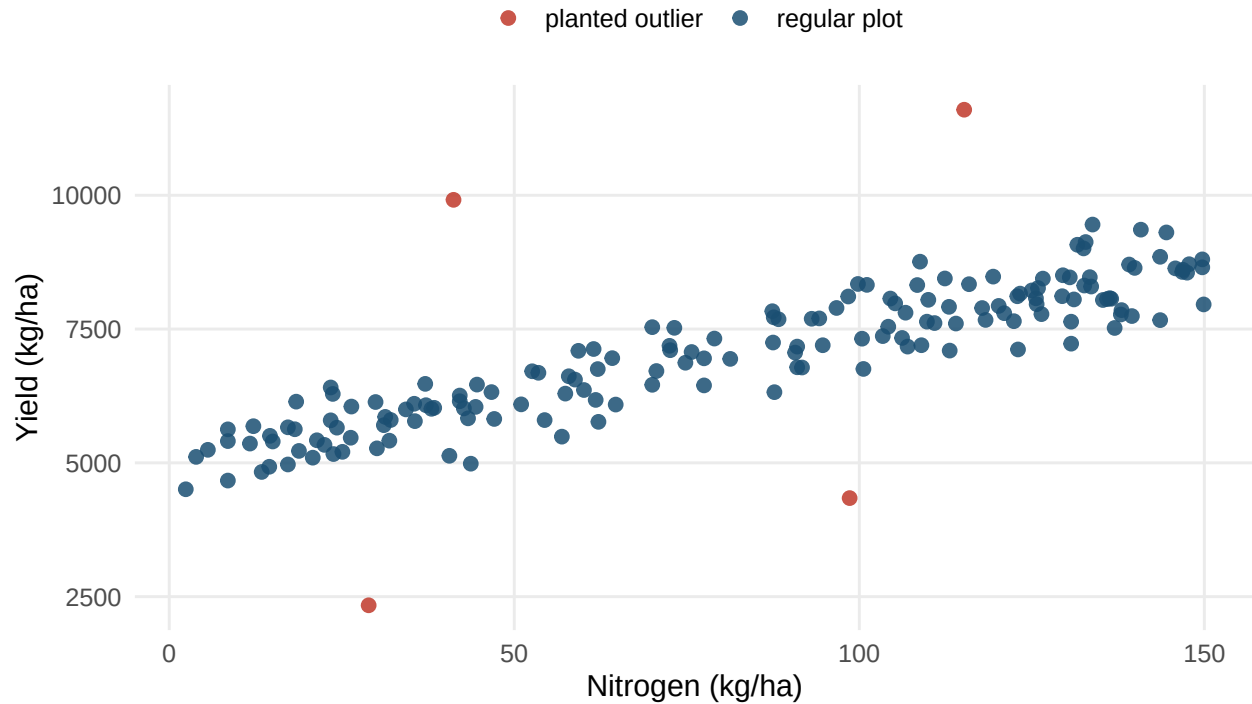


Figure 3.1: Simulated field trial: yield against nitrogen, with the five planted outliers highlighted.

i Interpretation

The cloud shows the expected relationship (more nitrogen, more yield) and also that the **spread grows** with nitrogen: this is the planted heteroscedasticity ($100 + 1.5 * \text{nitrogen}$). The five red points do not follow the cloud — two below, three above. No serious method should ignore this without at least declaring what it did.

```
na_block <- aggregate(cbind(nitrogen = is.na(field$nitrogen),
                           organic_matter = is.na(field$organic_matter)) ~ block,
                     data = field, FUN = mean)
na_long <- data.frame(
  block = rep(na_block$block, 2),
  variable = rep(c("nitrogen", "organic_matter"), each = nrow(na_block)),
  fraction = c(na_block$nitrogen, na_block$organic_matter)
)
ggplot(na_long, aes(block, fraction, fill = variable)) +
  geom_col(position = "dodge", width = .7) +
  scale_fill_manual(values = c(nitrogen = cores_smf[["primaria"]],
                              organic_matter = cores_smf[["destaque"]])) +
  labs(x = "block", y = "fraction missing", fill = NULL,
       title = "Missingness pattern by block") +
  theme(legend.position = "top")
```

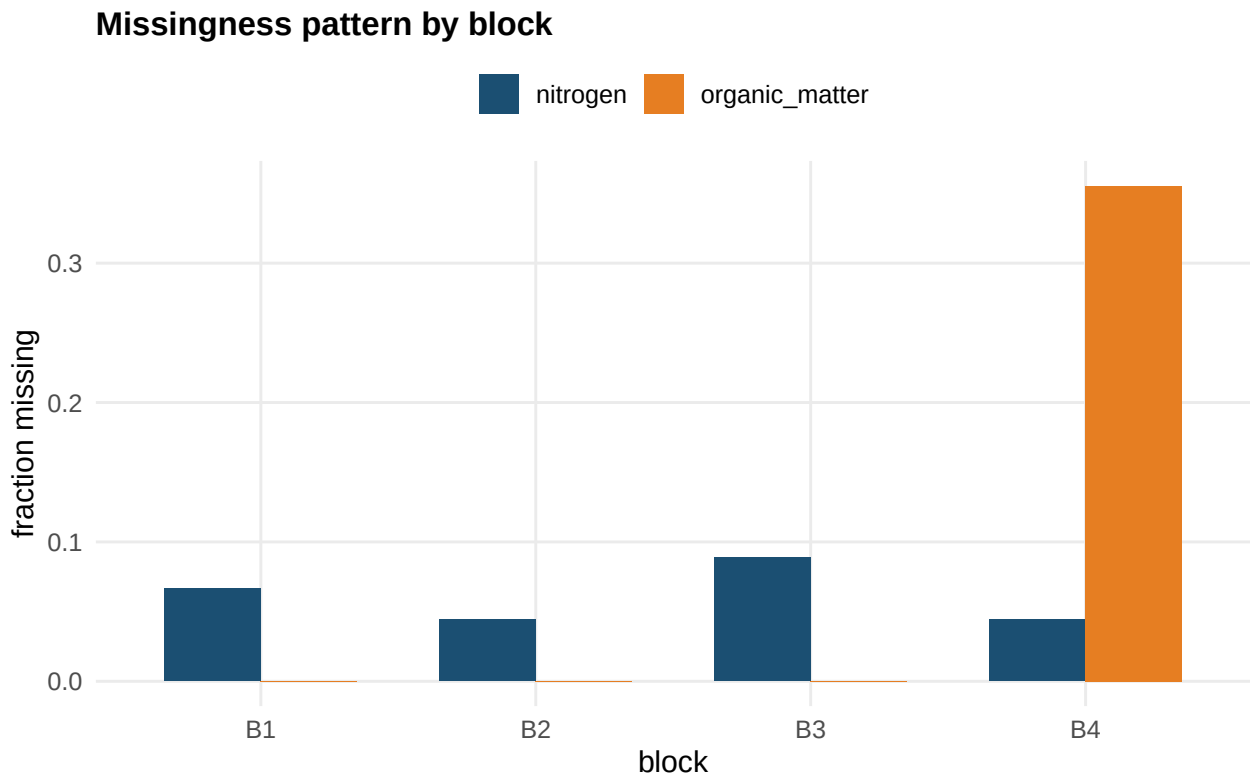


Figure 3.2: Missingness by block: nitrogen is missing at random (MCAR), but organic matter is missing more in block B4 (MAR).

⚠ Trap

The two missingness patterns **look** like the same problem and are not. Ignoring the MAR pattern (missingness that depends on the block) biases the mean of organic matter precisely in the block where it is most missing. Keep this figure in mind: it returns in Part 2, when we compare imputation strategies.

3.2 Two outcomes: an imbalanced class and a superdispersed count

```
disease <- ler("disease_binary")
prev <- mean(disease$disease)
cat("plants:", nrow(disease), "| observed prevalence:",
    fmt(prev, 3), "| flipped labels:", length(verdade$disease_binary$flip_ids), "\n")
```

plants: 400 | observed prevalence: 0.138 | flipped labels: 20

```
class_tbl <- data.frame(
  class = c("healthy (0)", "diseased (1)"),
```

```

n = as.integer(table(disease$disease)),
fraction = as.numeric(prop.table(table(disease$disease)))
)
ggplot(class_tbl, aes(class, n, fill = class)) +
  geom_col(width = .6, show.legend = FALSE) +
  geom_text(aes(label = paste0(n, " (", fmt(100 * fraction, 1), "%)")), vjust = -0.4) +
  scale_fill_manual(values = c(cores_smf[["primaria"]], cores_smf[["alerta"]])) +
  labs(x = NULL, y = "plants",
       title = "Imbalanced binary outcome") +
  ylim(0, max(class_tbl$n) * 1.15)

```

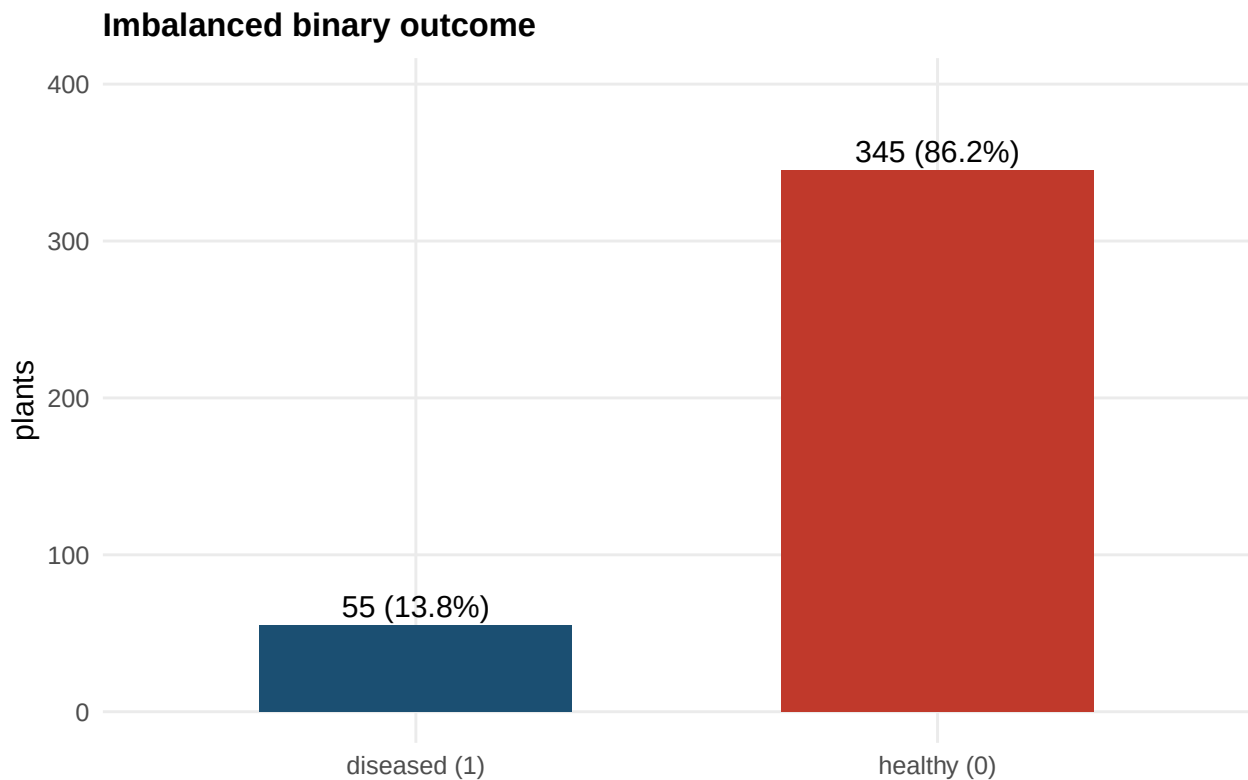


Figure 3.3: Class imbalance: only about 10% of plants are positive, and 5% of the labels were deliberately flipped.

⚠️ Trap

With 10% positives, **accuracy is a misleading metric**: predicting “healthy” for every plant is right about 86.2% of the time and learns nothing. On top of that, 5% of labels were flipped: there is a **performance ceiling** below 100%. Parts 3 and 4 show how to measure and communicate that ceiling.

3.3 Structure: groups, time, space and high dimension

```
farms <- ler("farm_groups")
ggplot(farms, aes(stats::reorder(farm, yield, FUN = median), yield)) +
  geom_boxplot(fill = cores_smf[["claro"]], outlier.size = .8) +
  labs(x = "farm (ordered by median)", y = "yield (kg/ha)",
       title = "Clustered data: farm effects") +
  theme(axis.text.x = element_text(size = 7, angle = 90, vjust = .5))
```

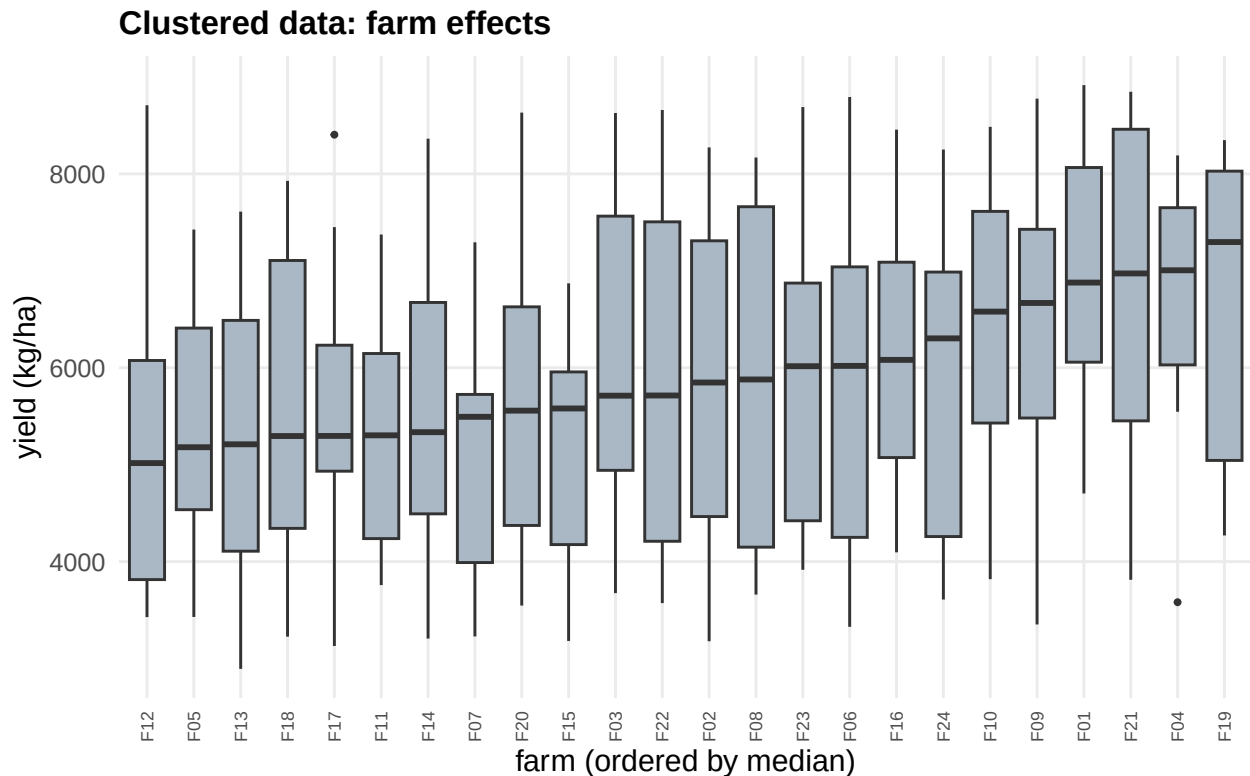


Figure 3.4: Clustering: 24 farms with 10 plots each; the average yield of each farm shifts the whole cloud.

```
climate <- ler("climate_series")
climate$shock <- climate$day %in% verdade$climate_series$shock_ids
ggplot(climate, aes(date, temperature)) +
  geom_line(colour = cores_smf[["neutro"]], linewidth = .4) +
  geom_point(data = subset(climate, shock), colour = cores_smf[["alerta"]], size = 2.4) +
  labs(x = NULL, y = "temperature (C)",
       title = "Time series: trend, season and shocks") +
  theme(legend.position = "none")
```

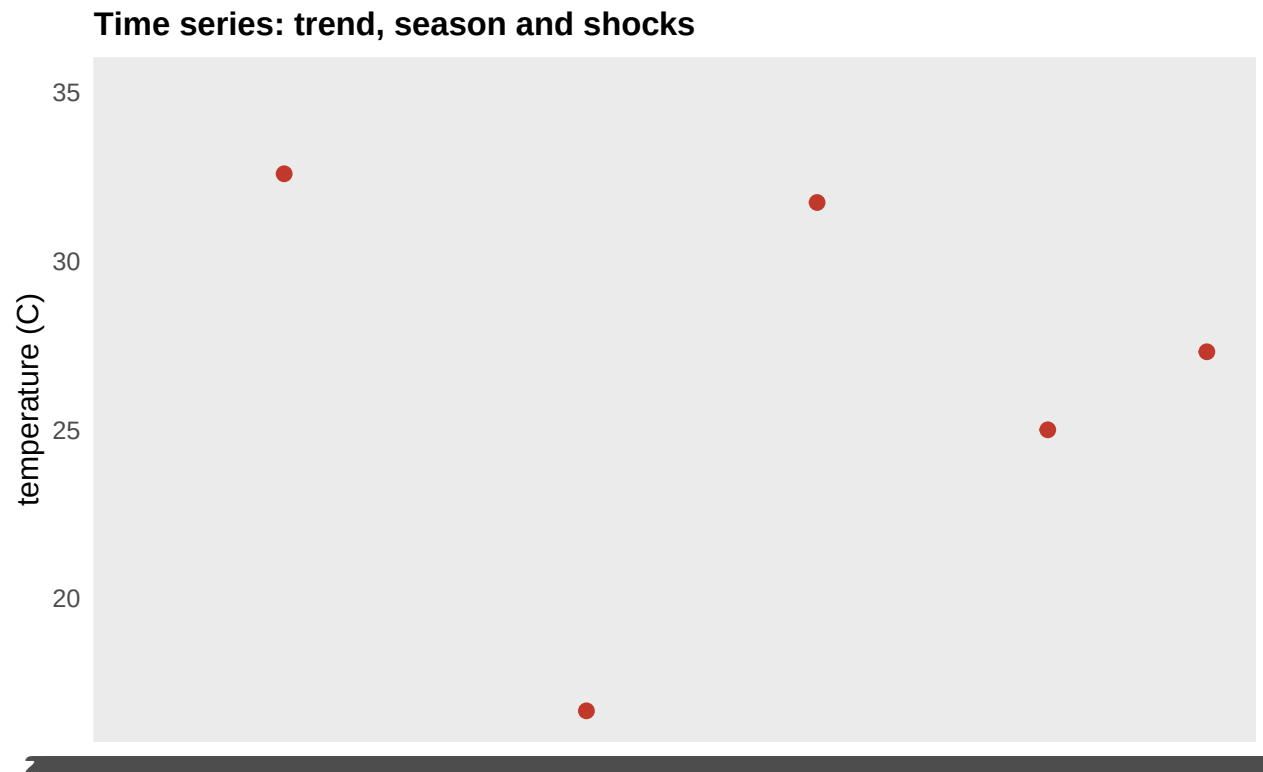


Figure 3.5: Simulated time series: trend, seasonality, autocorrelation and five planted shocks (red).

```
soil <- ler("soil_spatial")
ggplot(soil, aes(x, y, colour = organic_matter)) +
  geom_point(size = 2.1) +
  scale_colour_gradient(low = "#f6e8c3", high = cores_smf[["primaria"]],
                        name = "OM (%)") +
  coord_equal() +
  labs(x = "x (field)", y = "y (field)",
       title = "Spatial autocorrelation: organic matter") +
  theme(legend.position = "right")
```

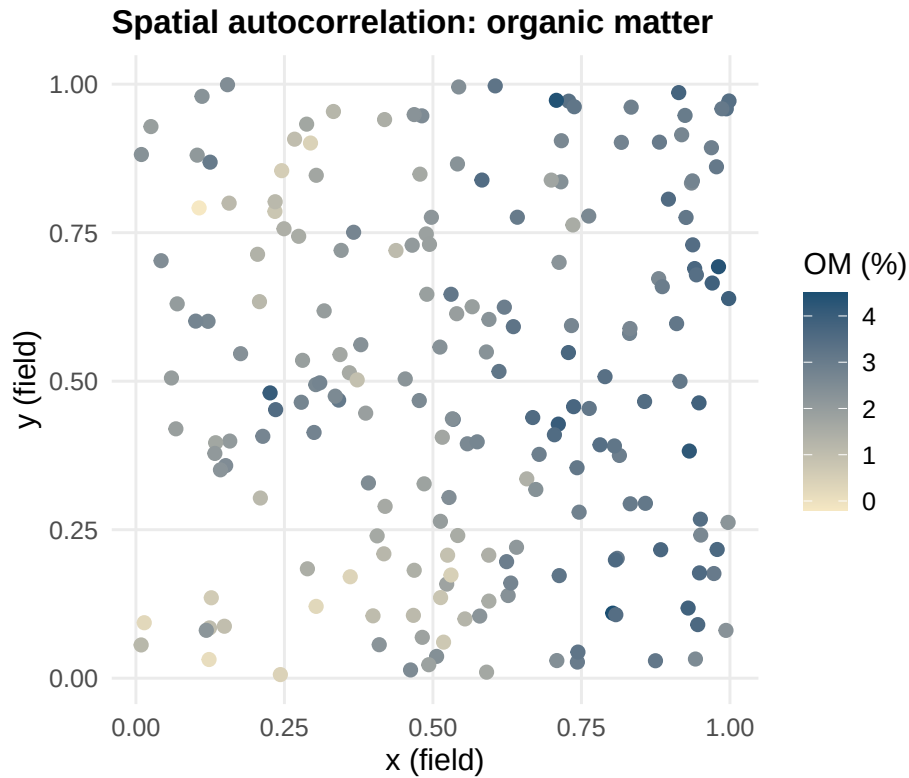


Figure 3.6: Spatial autocorrelation: organic matter at 200 points; neighbours look alike (range 0.25 on the field scale).

```
spectral <- ler("spectral_pn")
waves <- as.numeric(sub("w", "", grep("^w", names(spectral), value = TRUE)))
matrix_sp <- as.matrix(spectral[, grep("^w", names(spectral))])
sel <- c(1:7, 11, 47, 83, 109)
long_sp <- do.call(rbind, lapply(sel, function(i) {
  data.frame(lambda = waves, absorbance = matrix_sp[i, ],
             sample = spectral$sample_id[i],
             outlier = spectral$sample_id[i] %in% verdade$spectral_pn$outlier_ids)
}))
ggplot(long_sp, aes(lambda, absorbance, group = sample, colour = outlier)) +
  geom_line(alpha = .75, linewidth = .35) +
  scale_colour_manual(values = c(`FALSE` = cores_smf[["claro"]],
                                `TRUE` = cores_smf[["alerta"]]),
                     labels = c("regular (sample)", "outliers (4)")) +
  labs(x = "wavelength (nm)", y = "absorbance", colour = NULL,
       title = "High dimension: p >> n") +
  theme(legend.position = "top")
```

High dimension: $p \gg n$

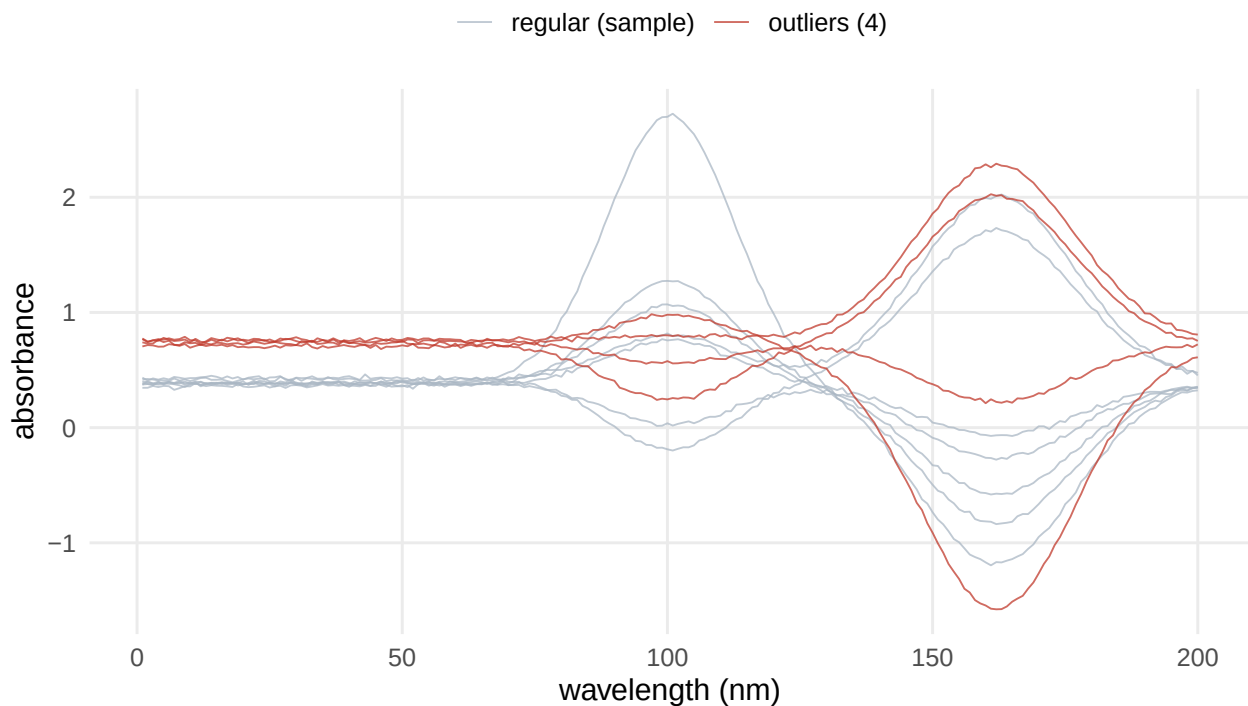


Figure 3.7: High dimension: 200 wavelengths for 120 samples; four spectra had their baseline shifted (red).

```
grid <- expand.grid(x1 = seq(-1, 1, by = .04), x2 = seq(-1, 1, by = .04))
grid$y <- sin(2 * pi * grid$x1) + 0.5 * grid$x2^2 + 0.8 * grid$x1 * grid$x2
ggplot(grid, aes(x1, x2, fill = y)) +
  geom_raster() +
  scale_fill_viridis_c(option = "C", name = "y") +
  coord_equal() +
  labs(x = "x1", y = "x2",
       title = "Deep learning target: nonlinear surface") +
  theme(legend.position = "right")
```

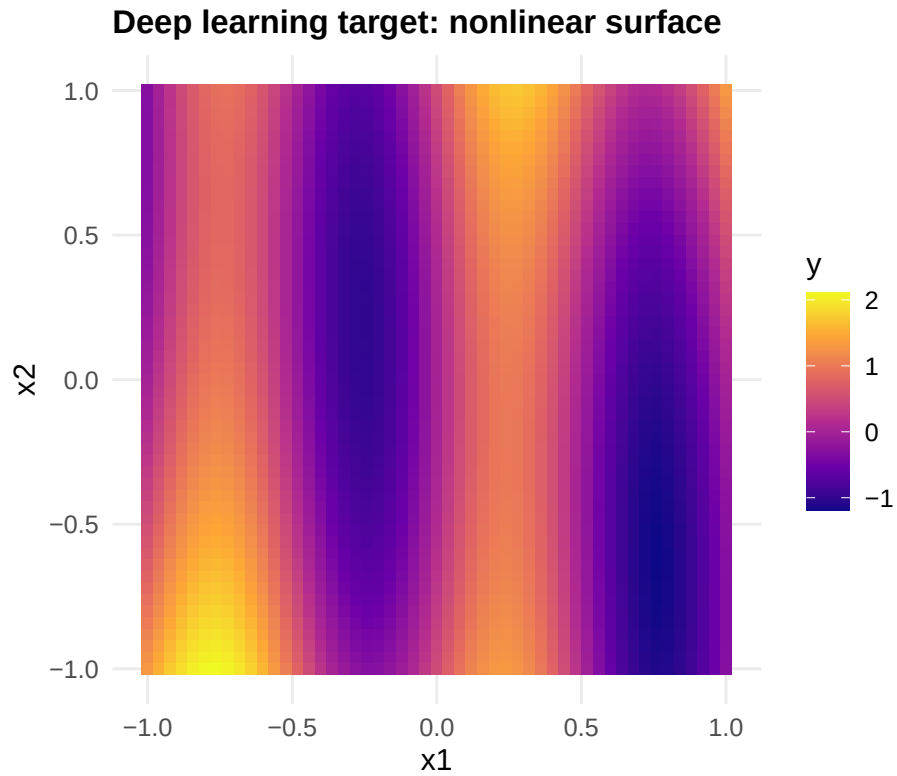


Figure 3.8: Planted nonlinearity for Part 6: the true surface combines sine, square and interaction.

The count dataset (`pest_counts`), the small sample (`bayes_small`) and the classification dataset enter their corresponding parts, with the same rules: discrepancy table, real execution and comparison against the truth.

4 Audit and contracts

Before any model, three declarations: **what** is to be predicted, **which** variables enter and **how** the observations relate. In the package, each is an object.

```
task <- smf_task_spec("regression", "yield")
data_spec <- smf_data_spec(
  target = "yield",
  predictors = c("nitrogen", "potassium", "water", "organic_matter"),
  id_column = "plot_id",
  units = c(yield = "kg/ha", nitrogen = "kg/ha", water = "mm"),
  categorical = c("block", "genotype")
)
design_spec <- smf_design_spec(id_column = "plot_id", block_columns = "block")
print(task); print(design_spec)
```

```
<sciModelFlowR::TaskSpec>
@ kind      : chr "regression"
@ target    : chr "yield"
@ positive_label: NULL
@ objective  : chr(0)

<sciModelFlowR::DesignSpec>
@ id_column      : chr "plot_id"
@ group_columns  : chr(0)
@ block_columns  : chr "block"
@ repeated_unit  : chr(0)
@ time_column    : chr(0)
@ coordinate_columns : chr(0)
@ external_domain_column: chr(0)
@ hierarchy      : list()
@ experimental_unit : chr(0)
```

The audit walks the dataset and returns what must be seen **before** modeling: missing values per variable, duplicates, cardinality, leakage signals and quality flags.

```
audit <- smf_audit_data(field, data_spec, design_spec)
tab_smf(subset(audit@missingness, n_missing > 0),
  caption = "Audit: missing values found in the field trial.")
```

Table 4.1: Audit: missing values found in the field trial.

	variable	n_missing	fraction_missing
4	nitrogen	11	0.061
7	organic_matter	16	0.089

```
cat("duplicates (rows):", length(audit@duplicates$rows),
    "| quality flags:", length(audit@quality_flags),
    "| leakage signals:", length(audit@leakage), "\n")
```

duplicates (rows): 0 | quality flags: 0 | leakage signals: 0

```
ggplot(audit@cardinality, aes(stats::reorder(variable, n_unique), n_unique)) +
  geom_col(fill = cores_smf[["primaria"]], width = .7) +
  geom_text(aes(label = n_unique), hjust = -0.2, size = 3) +
  coord_flip() +
  labs(x = NULL, y = "unique values",
       title = "Cardinality audit") +
  ylim(0, max(audit@cardinality$n_unique) * 1.12)
```

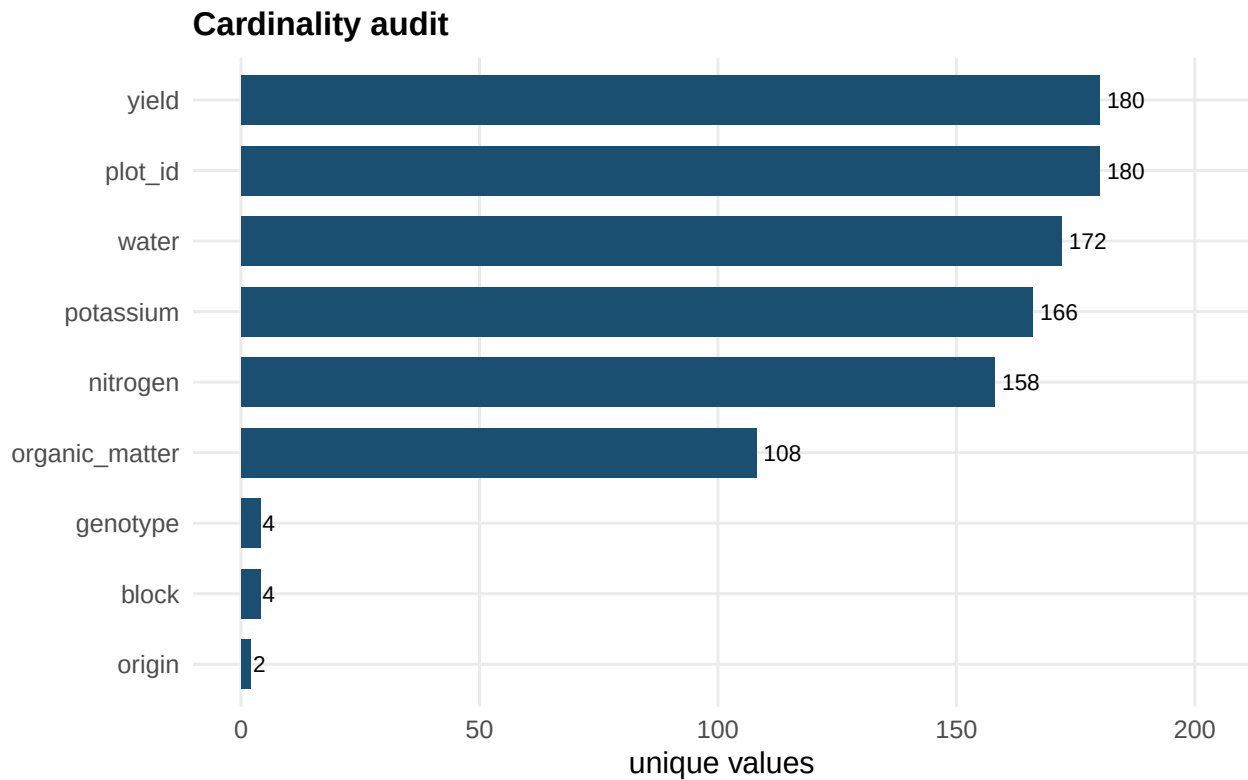


Figure 4.1: Cardinality per variable: unique identifiers, repeated blocks and genotypes, continuous doses.

i Interpretation

The audit does **not** look for outliers: it covers structure (missingness, duplicates, cardinality, leakage). Detecting numerical discrepancies is analysis, not audit — and that is what we do next, by hand, to learn the criterion.

4.1 Finding the planted outliers

A simple and robust criterion is the **robust z-score**: distance from the median divided by the median absolute deviation (MAD). Its advantage over the classical z is that the outlier itself does not inflate the ruler.

```
robust_z <- (field$yield - stats::median(field$yield, na.rm = TRUE)) /
  stats::mad(field$yield, na.rm = TRUE)
field$robust_z <- robust_z
field$flag_z <- abs(robust_z) > 3.5
planted <- field$plot_id %in% verdade$field_yield$outlier_ids
confusion <- table(planted = planted, flag_z = field$flag_z)
tab_smf(as.data.frame.matrix(confusion),
  caption = "Detection by robust z (> 3.5) against the planted outliers.")
```

Table 4.2: Detection by robust z (> 3.5) against the planted outliers.

	FALSE
FALSE	175
TRUE	5

```
ggplot(field, aes(seq_along(robust_z), robust_z)) +
  geom_point(aes(colour = origin), size = 2) +
  geom_hline(yintercept = c(-3.5, 3.5), linetype = "dashed", colour = cores_smf[["neutro"]])
  scale_colour_manual(values = c("regular plot" = cores_smf[["primaria"]],
    "planted outlier" = cores_smf[["alerta"]])) +
  labs(x = "file position (not the id)", y = "robust z of yield",
    colour = NULL,
    title = "Robust z-score screening") +
  theme(legend.position = "top")
```

Robust z-score screening



Figure 4.2: Robust z per plot: the ruler built from median and MAD separates the five planted points from the rest.

! Interpretation

The ruler flagged 0 plots. Of the five planted ones, 0 were detected and 0 false alarm(s) appeared. Detection is not judgement: the next step is to **investigate** (typing error? unusual plot? treatment?) and **declare** the decision. In Part 3 we will see the effect of these decisions on the fit and on the residuals.

4.2 Contracts: what the package refuses

The package turns scientific rules into execution errors. Four invalid attempts, executed now:

```
try_it <- function(expr) tryCatch(expr, error = function(e) conditionMessage(e))
r1 <- try_it(smf_data_spec("yield", c("nitrogen", "yield"), "plot_id"))
r2 <- try_it(smf_fit_preprocessor(field, smf_preprocess_spec("median"),
                                data_spec, role = "test"))
r3 <- try_it(smf_apply_imbalance(field, "yield",
                                smf_imbalance_spec("downsample"),
                                context = "test", seed = 20260919))
r4 <- try_it(smf_bootstrap(farms, smf_bootstrap_spec("case", n_resamples = 49L),
                          statistic = function(z) c(mean = mean(z$yield)),
```

```

design = smf_design_spec(id_column = "plot_id",
                        group_columns = "farm")))

refusals <- data.frame(
  attempt = c("target also used as predictor",
             "preprocessing fitted with role = test",
             "balancing applied with context = test",
             "iid bootstrap on data with declared groups"),
  response = c(r1, r2, r3, r4)
)
names(refusals) <- c("Invalid attempt", "Message returned by the package")
tab_smf(refusals, caption = "Contracts verified at execution time.")

```

Table 4.3: Contracts verified at execution time.

Invalid attempt	Message returned by the package
target also used as predictor - @target cannot also be listed in @predictors	object is invalid:
preprocessing fitted with role = test	Managed preprocessing can only be fitted on training data.
balancing applied with context = test	Sampling or class-weight estimation must not be learned/applied from assessment or final test labels.
iid bootstrap on data with declared groups	The requested bootstrap unit does not preserve dependence declared in DesignSpec.

Interpretation

The four refusals are the same rule in four places: **evaluation information does not take part in building the model, and the resampling unit respects the declared dependence.** A package that accepted any of these calls would return optimistic numbers that look normal.

In your experiment

Start every project with three questions: (1) what is the experimental unit and what is the identifier? (2) is there declared clustering (farm, block, plot), time or space? (3) which columns are “evaluation” columns that must never touch training? The answers become `smf_data_spec()` and `smf_design_spec()`.

5 Part 1 exercises

Exercise 1.1 — MCAR versus MAR

In the field trial, show numerically that the missingness of `nitrogen` does not depend on the block, while that of `organic_matter` does. Which of the two imputation strategies do you suspect will suffer more? Save the answer for Part 2.

Exercise 1.2 — Another detection ruler

Detect the `yield` outliers with the interquartile range rule ($1.5 \times \text{IQR}$ beyond the quartiles) and compare with the robust `z`: which finds more planted points, which errs more? Why is the IQR even more fragile than the MAD?

Exercise 1.3 — Auditing another dataset

Build `smf_task_spec()`, `smf_data_spec()` and `smf_design_spec()` for the `disease_binary` dataset (target `disease`; predictors `x1` to `x4`; id `plant_id`) and run `smf_audit_data()`. What does the audit show about prevalence? Does it “see” the imbalance? And the label noise?

Exercise 1.4 — Why was the iid bootstrap refused?

Explain, in two sentences, what would happen to the confidence interval of the mean `yield` in `farm_groups` if each plot were treated as independent. (Hint: between-farm variance is 300 and residual variance is 120.)

Solutions

1.1. The missingness fraction of `nitrogen` per block is essentially constant (MCAR), while `organic_matter` concentrates missingness in block B4 (MAR). Global-mean imputation should suffer more under MAR, because the mean of block B4 differs from the others; imputation that respects the block (or a model including `block`) reduces the bias.

1.2. The IQR rule tends to flag more points on the long tail created by heteroscedasticity (high yields at high doses), mixing “discrepant” with “expected”. The MAD is more stable because the median and the MAD barely move under contamination.

1.3. The audit returns the missing counts (zero), duplicates (zero) and cardinality; imbalance shows up only as **cardinality** of the target (two unique values), and label noise is not detectable by audit — no automatic procedure knows which label was flipped.

1.4. Treating plots as independent ignores the between-farm variance (sd 300 against residual 120), producing intervals that are **too narrow** and conclusions that are too confident. The correct resampling unit is the farm.

```
cat("Part 1 finished at", format(Sys.time(), "%Y-%m-%d %H:%M"),  
    "| package", as.character(packageVersion("sciModelFlowR")), "\n")
```

Part 1 finished at 2026-09-19 00:46 | package 1.0.2